

大問 3

(概要)

迷路の解を求めるプログラム。迷路の解を求めるアルゴリズムは大体、スタックを用いた幅優先探索かキューを用いた深さ優先探索という予想がついているとプログラムの内容理解がスムーズ

問 1(スタックについての説明)

(解)

スタックとは最後に入れられたデータが一番最初に取り出される(LIFO, Last In First Out)データ構造のこと。スタックにデータを登録する操作のことを **push**、取り出す操作のことを **pop** という。

(補足)

キューのデータ構造は最初に入れられたデータが最初に取り出される(FIFO, First In First Out)構造。

問 2(point の型定義推定)

point とは現在の探索する座標を示す構造体。つまり **x** 座標と **y** 座標を持つ。

(解)

```
typedef struct{
    int x;
    int y;
}point;
```

(補足)

構造体のポインタからメンバ変数へのアクセスは->(アロー演算子)を用いる。

問 3(maze の構造推定)

maze は迷路の構造を示す変数であり、2次元配列で示される。この2次元配列の中に壁(**wall**)や道(**path**)の情報が入っており、壁は1、道は0で表現されている。

(解)

```
int maze[HEIGHT][WIDTH];
wall...1
path...0
```

問 4(配列 dx、dy の推定)

for 文を確認すると **dx**、**dy** はそれぞれ4つの要素を持ち、現在の座標(**p->x**、**p->y**)に加えたものをデータ構造に入れていることから、配列 **dx**、**dy** は現在の座標から次に探索する座標への方向を示すものであると予想できる。迷路における次の探索座標は4方向の左右上下である。

(解)

```
int dx[4] = { 0,0,-1,1};
int dy[4] = {-1,1, 0,0};
```

問 5(プログラムの構造理解)

登録関数が全体で1度だけしか呼び出されなかったということは、for ループ内の1ループ目で **if(p == NULL)**または **if(maze[p->y][p->x] != 0)**、**if(p->x == GoalX && p->y == GoalY)**の条件文に引っかかったということである。

(解)

1, 4

問 6(プログラムの構造理解、探索アルゴリズムの理解)

迷路の探索範囲を調べて、探索範囲にゴールが存在できるかを調べる。迷路の形状が左右上下に対称であるからスタート位置を右上側にあるものとし、探索は左右上下の順に探索するものとする。

↑	←	↑	←	↑	↑	→
↑	↓	↑	↓	S	↑	↓
↑	↓	↑	↓	↑	↑	↓
↑	↓	↑	↓	↑	↑	↓
↑	↓	↑	↓	↑	↑	↓
↑	↓	↑	↓	↑	↑	↓
←	↓	←	↓	→	→	↓

スタックを用いた探索の場合

↑	↑	↑	↑	↑	↑	↑
←	←	←	←	S	→	→
↓	↓	↓	↓	↓	↓	↓
↓	↓	↓	↓	↓	↓	↓
↓	↓	↓	↓	↓	↓	↓
↓	↓	↓	↓	↓	↓	↓
↓	↓	↓	↓	↓	↓	↓

キューを用いた探索の場合

これらからゴールが迷路内のどの位置にあっても探索可能であることがわかる。

(解)

2

問 7(探索アルゴリズムの理解)

以下のような迷路の一部を例として挙げる。説明の簡易化のために座標は左上端のマスを(1,1)とする。

また、実際には進めない座標も1度はスタックに push されるが処理を飛ばされるのでここでは進めない座標は push されないように扱う。

1	-1	1	1	0	0	0
1	-1	1	1	1	0	1
1	-1	1	1	1	0	1
1	-1	0	0	0	0	1
1	0	1	1	1	0	1
1	0	0	0	1	0	0
1	1	1	1	1	1	1

今(2,4)でスタックに(3,4)と(2,5)を push したところである。
スタックには今 {(3,4),(2,5)} が積まれており、次に取り出されるのは(2,5)である。
(2,5)を pop し、処理を実行する。(2,6)が push されスタックは{(3,4),(2,6)}となる。
次に(2,6)が pop され、(3,6)が push される。スタックは{(3,4),(3,6)}になる。
次に(3,6)が pop され、(4,6)が push される。スタックは{(3,4),(4,6)}になる。
(4,6)が pop されると、進める座標がないので座標を push せず、スタックは{(3,4)}となる。
すると(3,4)が pop され、探索が再開される。
このように行けるところまで行き、行けなくなったら戻ってまた探索をすることを深さ優先探索という。

また、キューを用いた処理では、{(3,4),(2,5)}となっているとき、次に取り出されるのは(3,4)である。
(3,4)を dequeue し、処理を実行する。(4,4)が enqueue され、キューは{(2,5),(4,4)}となる。
次に(2,5)が dequeue され、(2,6)が enqueue される。キューは{(4,4),(2,6)}となる。
次に(4,4)が dequeue され、(4,5)が enqueue される。キューは{(2,6),(4,5)}となる。
このようにスタートから近い場所から順に探索することを幅優先探索という。
幅優先探索の場合、複数の道を同時に探索するため分かれ道が多い迷路では深さ優先探索よりも時間がかかってしまうことが多く、分かれ道の座標をキューに記憶する分メモリを消費しやすい。
その一方でゴールにたどり着いた座標を遡っていくことで常に最短経路を求めることができる。

(解)

スタックを用いた実装の場合は深さ優先探索となり、キューを用いた実装の場合、幅優先探索となる。幅優先探索の場合、分かれ道が多い迷路では深さ優先探索よりも時間がかかったりメモリを消費したりする。

コメントを付けたプログラムを以下に示す。

```
point *p;    //座標を示すポインタ
int d;       //ループ変数

登録(StartX, StartY);    //スタート座標をデータ構造に追加

//for ループ
for( ; ; ){
    p = 所得();           //データ構造から探索する座標を取り出す
    if(p == NULL){       //探索する座標がもうなければ
        printf("解は存在しません\n");
        exit(1);         //終了
    }
    //取り出した座標が未知の道以外なら次の探索座標へ
    if(maze[p->y][p->x] != 0) continue;

    //取り出した座標がゴール座標なら for ループを抜けて終了
    if(p->x == GoalX && p->y == GoalY) break;

    //探索する座標を既知の道とする
    maze[p->y][p->x] = -1;

    //次に探索する座標をデータ構造に追加
    for(d = 0; d < 4; d++){
        登録(p->x + dx[d], p->y + dy[d]);
    }
}
printf("解は存在します\n");
```